



PEC3 - Tercera prueba de evaluación continua

Presentación

Esta PEC se focaliza en los circuitos secuenciales. Los circuitos combinacionales nos permiten describir funcionalidades de un circuito pero no nos permiten guardar información. Mediante biestables y registros podemos guardar información en memoria y hacer circuitos más complejos. En esta PEC practicaremos con este tipo de circuitos.

Competencias

- Entender el funcionamiento de los circuitos lógicos secuenciales y conocer y saber aplicar técnicas de diseño de sistemas secuenciales.

Objetivos

- Saber discernir, a partir de la funcionalidad que se quiere que tenga un circuito lógico, si el circuito tiene que ser de tipo secuencial o combinacional.
- Conocer el funcionamiento del biestable D y de todas las entradas de control que puede tener.
- Saber analizar un circuito secuencial.
- Saber realizar un cronograma a partir de un circuito digital secuencial.
- Saber analizar un grafo de estados.
- Saber diseñar un circuito cualquiera a partir de la descripción de su funcionalidad mediante el modelo de Moore.

Recursos

Los recursos que se recomienda usar para esta PEC son los siguientes:

Básicos: El módulo 4 de los materiales.

Complementarios: VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.



Uso de herramientas de IA

En esta actividad no está permitido el uso de herramientas de inteligencia artificial. En el plan docente y en la [web sobre integridad académica y plagio](#) de la UOC encontraréis información sobre que se considera conducta irregular en la evaluación y las consecuencias que puede tener.

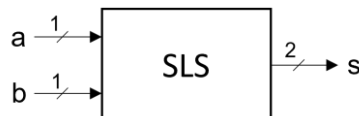
Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas proporcionadas junto con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega de la Actividad** correspondiente del apartado **Contenidos** de vuestra aula.
- La fecha límite de entrega es el **8 de mayo** (a las 24 horas).

Solución

Ejercicio 1 [25%]

Se quiere diseñar el grafo de estados de un circuito lógico secuencial llamado SLS. El circuito tiene dos entradas de un bit cada una, denominadas a y b , y una salida de dos bits, denominada s , según la estructura siguiente:



El circuito SLS detecta si la entrada a es igual que la entrada b durante 3 ciclos consecutivos, o bien si la entrada a es diferente de la entrada b durante dos ciclos consecutivos. Cuando el circuito detecta que $a = b$ durante 3 ciclos, en el ciclo siguiente pone el valor 01 en la salida s durante un ciclo. Similarmente, cuando el circuito detecta que $a \neq b$ durante 2 ciclos, en el ciclo siguiente pone el valor 10 en la salida s durante un ciclo. Al empezar a funcionar el circuito, la salida s vale 00.

Si $a = b$ durante más de 3 ciclos, la salida s se mantendrá a 01 mientras a y b sean iguales. Similarmente, si $a \neq b$ durante más de 2 ciclos, la salida se mantendrá a 10. En cualquier otro caso la salida valdrá 00.

Ejemplo de funcionamiento del circuito:

Entrada a	0	1	1	0	1	1	0	1	0	1	1	...
Entrada b	1	1	1	0	1	0	0	0	1	1	0	...
Salida s	00	00	00	00	01	01	00	00	00	10	00	...

Se pide que diseñéis el grafo de estados del circuito SLS, especificando claramente su leyenda y su estado inicial.



Partiremos de un estado inicial, que denominaremos INI, en el que las salidas valdrán 00. Lo que tendremos que distinguir en cada estado es si las entradas a y b son iguales o diferentes, de forma que de todos los estados saldrán dos arcos, uno para los valores de entrada 00 o 11 y otro para los valores de entrada 01 o 10.

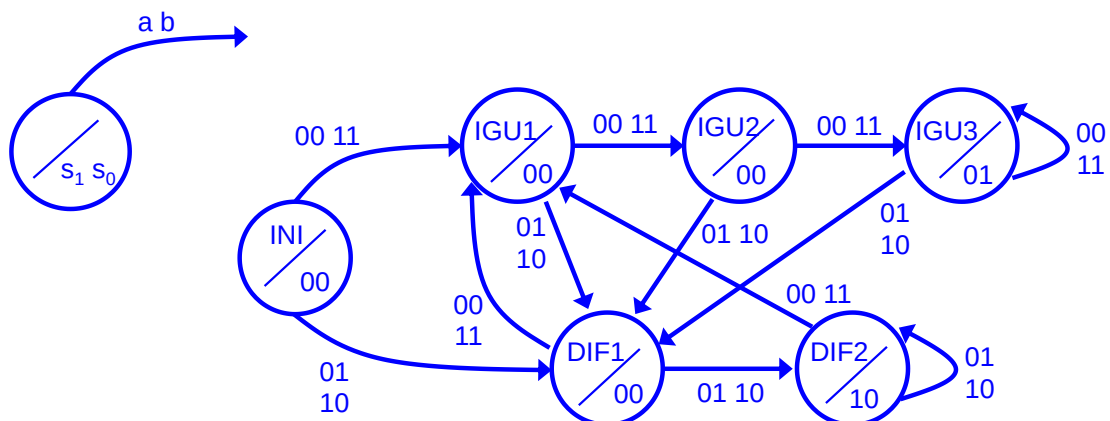
Desde el estado INI, si las entradas son iguales iremos a un estado llamado IGU1, y si en los ciclos siguientes a y b siguen siendo iguales pasaremos a los estados IGU2 e IGU3. En este estado las salidas valdrán 01, y nos quedaremos en él mientras se siga cumpliendo $a = b$.

Análogamente, si estando en INI llegan valores diferentes en las entradas a y b pasaremos en un estado con nombre DIF1. Si en el ciclo siguiente a y b siguen siendo diferentes pasaremos al estado DIF2, en el que las salidas valdrán 10, y nos quedaremos en él mientras se siga cumpliendo $a \neq b$.

En todos los estados que no sean IGU3 ni DIF2 las salidas valdrán 00.

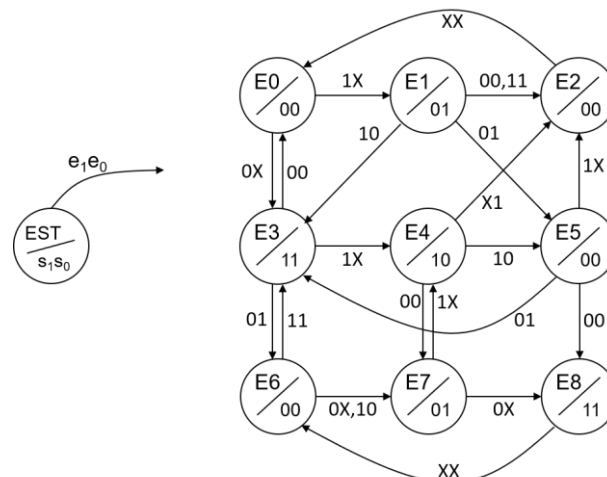
Estando en cualquiera de los estados IGU i , si llegan valores diferentes en las entradas pasaremos al estado DIF1. Del mismo modo, si estando en cualquiera de los estados DIF i llegan valores iguales en las entradas, pasaremos al estado IGU1.

El grafo de estados completo se muestra a continuación.



Ejercicio 2 [25%]

Dado el grafo de estados siguiente:





Se pide:

- a) **[5%]** ¿Cuántos bits de entrada tiene el circuito que implementa este grafo? ¿Cuántos bits de salida? ¿Cuál será el número mínimo de biestables para implementarlo? Si lo implementamos utilizando una memoria ROM, ¿cuántos bits de direcciones y cuántos bits de datos necesitará esta memoria?

El número de bits de entrada lo vemos, por ejemplo, en la leyenda: son **2 bits** (e_1 y e_0).

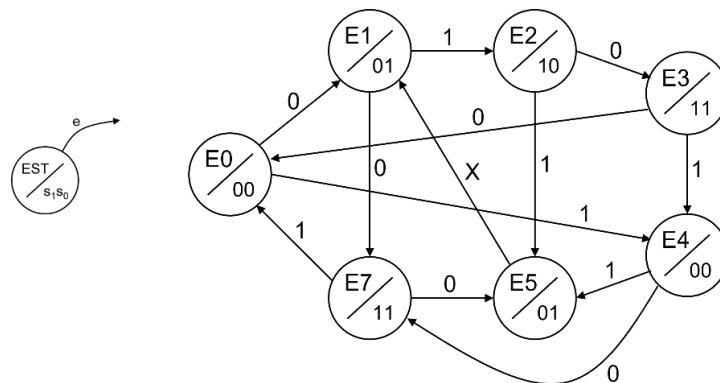
Igualmente vemos que el circuito tiene **2 bits de salida** (s_1 y s_0).

El grafo tiene 9 estados, y por tanto harán falta 4 bits para codificarlos ($\log_2(9)$ es un número real entre 3 y 4). De forma que el circuito tendrá que tener (al menos) **4 biestables**, uno para cada bit de codificación de los estados. Los podemos denominar q_3 , q_2 , q_1 y q_0 .

Al implementar un circuito secuencial con una memoria ROM, la ROM tiene que tener una palabra por cada estado y cada posible combinación de variables de entrada. Es decir, una palabra por cada combinación de las señales [q_3 q_2 q_1 q_0 e_1 e_0]. Por lo tanto, éstas son las señales que llegarán a la entrada de direcciones de la ROM, que será de **6 bits** (y la ROM tendrá $2^6 = 64$ palabras).

Cada palabra tiene que indicar el estado futuro (q_3^+ , q_2^+ , q_1^+ y q_0^+) y el valor de las 2 señales de salida en el estado actual. Por lo tanto, cada palabra tendrá $4 + 2 =$ **6 bits**.

Dado el grafo de estados siguiente:



Se pide:

- b) **[10%]** Escribid la tabla de transiciones y la tabla de salidas del sistema representado por el grafo, codificando los estados según su índice asociado. Al escribir la tabla de transiciones, poned en primer lugar las variables que codifican el estado y a continuación las variables de entrada.

Nota: Tenéis disponible el ejercicio en VerilCHART. Para poder probar este ejercicio en VerilCHART tenéis que sustituir, si fuera el caso, los bits *don't care* por valores 0.



Dado que el grafo tiene 7 estados, harán falta 3 variables para codificarlos. Tal y como es habitual, las denominamos q_2 q_1 y q_0 y definimos la codificación de los estados así: para codificar el estado de nombre E_i , daremos a $[q_2 \ q_1 \ q_0]$ el valor de i codificado en binario.

Para obtener la tabla de transiciones observamos en el grafo a qué estado se llega desde cada estado con cada valor posible de la variable de entrada. Puesto que el estado E6 no existe, las filas correspondientes de la tabla son combinaciones *don't care*.

q_2	q_1	q_0	e	q_2^+	q_1^+	q_0^+
0	0	0	0	0	0	1
0	0	0	1	1	0	0
0	0	1	0	1	1	1
0	0	1	1	0	1	0
0	1	0	0	0	1	1
0	1	0	1	1	0	1
0	1	1	0	0	0	0
0	1	1	1	1	0	0
1	0	0	0	1	1	1
1	0	0	1	1	0	1
1	0	1	0	0	0	1
1	0	1	1	0	0	1
1	1	0	0	x	x	x
1	1	0	1	x	x	x
1	1	1	0	1	0	1
1	1	1	1	0	0	0

Para rellenar la tabla de salidas basta con escribir el valor que tienen las variables de salida en cada estado.

q_2	q_1	q_0	s_1	s_0
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	x	x
1	1	1	1	1



- c) **[10%]** Diseñad el circuito que implementa el sistema representado por el grafo de estados utilizando una memoria ROM. Mostrad el contenido completo de la memoria ROM en hexadecimal.

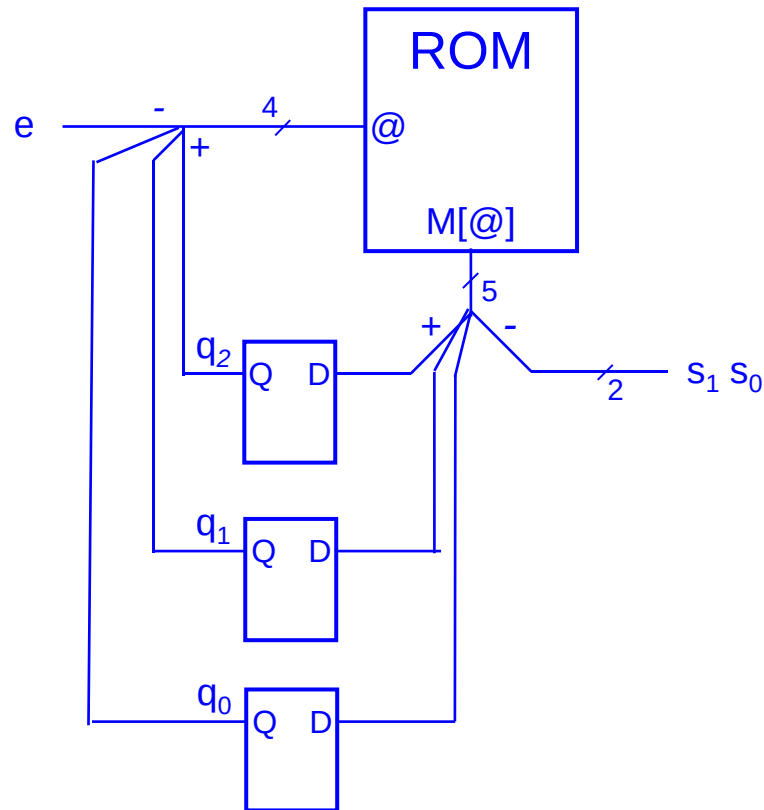
Nota: Tenéis disponible el ejercicio en VerilCIRC. Para poder probar este ejercicio en VerilCIRC tenéis que sustituir, si fuera el caso, los bits *don't care* por valores 0.

El circuito tendrá una memoria ROM con una entrada de direcciones de 4 bits (las tres variables que codifican el estado y la variable de entrada al circuito), y por lo tanto 16 palabras, que contendrán el valor del estado futuro ($d_i = q_i^+$ con $i = 0, 1, 2$) y el valor de las 2 variables de salida en cada estado. Es decir, las palabras serán de 5 bits.

A continuación se muestra el contenido de la ROM. Hemos dado el valor 0 a las condiciones *don't care*.

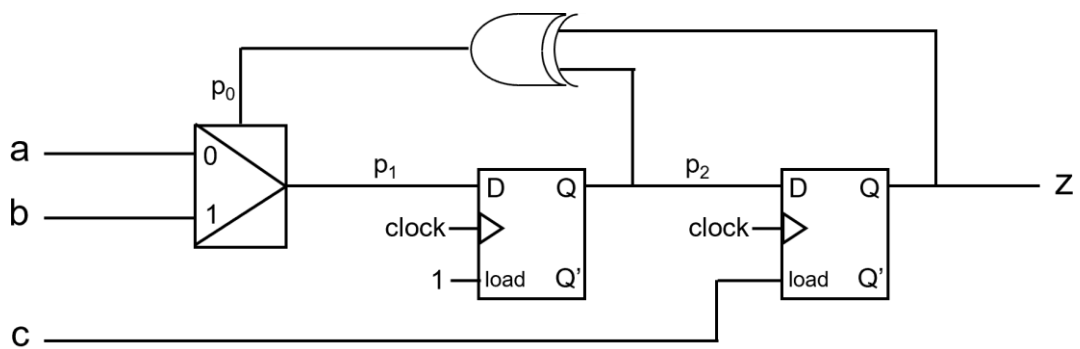
q_2	q_1	q_0	e	d_2	d_1	d_0	s_1	s_0	En hexadecimal
0	0	0	0	0	0	1	0	0	04h
0	0	0	1	1	0	0	0	0	10h
0	0	1	0	1	1	1	0	1	1Dh
0	0	1	1	0	1	0	0	1	09h
0	1	0	0	0	1	1	1	0	0Eh
0	1	0	1	1	0	1	1	0	16h
0	1	1	0	0	0	0	1	1	03h
0	1	1	1	1	0	0	1	1	13h
1	0	0	0	1	1	1	0	0	1Ch
1	0	0	1	1	0	1	0	0	14h
1	0	1	0	0	0	1	0	1	05h
1	0	1	1	0	0	1	0	1	05h
1	1	0	0	0	0	0	0	0	00h
1	1	0	1	0	0	0	0	0	00h
1	1	1	0	1	0	1	1	1	17h
1	1	1	1	0	0	0	1	1	03h

Además, el circuito tendrá 3 biestables, que guardarán el estado actual. A continuación se muestra el circuito completo.

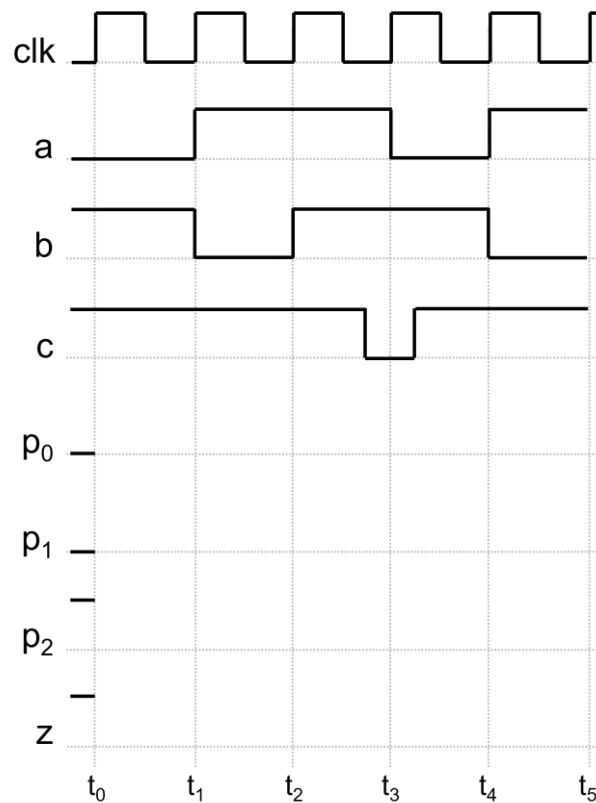


Ejercicio 3 [25%]

Dado el circuito siguiente:



Completa este cronograma:



Nota: Tenéis disponible este ejercicio en VerilCHART.

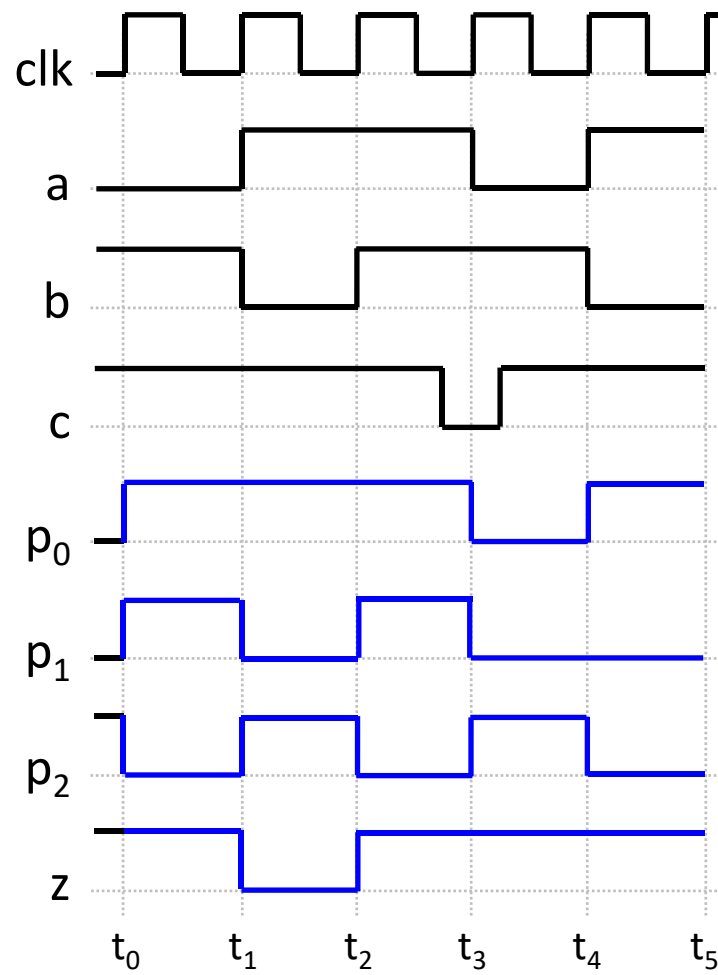
Analizamos los valores de cada señal a partir del circuito:

- $p_0 = p_2 \text{ XOR } z$
- p_1 vale a siempre que $p_0 = 0$, y b siempre que $p_0 = 1$.
- p_2 es la señal de salida de un biestable que se carga con p_1 a cada flanco ascendente de reloj. Por lo tanto, a cada flanco p_2 pasará a valer lo que valía p_1 antes del flanco.
- z es la señal de salida de un biestable que solo cambia de valor en los flancos en los que $c = 1$. El valor que se carga en el biestable en estos casos es p_2 .

Veamos qué pasará en el instante t_0 :

- p_2 pasará a valer 0, ya que es el valor que tenía p_1 antes del flanco.
- Puesto que hasta el flanco $c = 1$, z pasará a valer 1, que es lo que valía p_2 antes del flanco.
- En consecuencia, p_0 valdrá $0 \text{ XOR } 1 = 1$ entre t_0 y t_1 , y por lo tanto p_1 valdrá b , que se mantiene a 1 durante todo este intervalo.

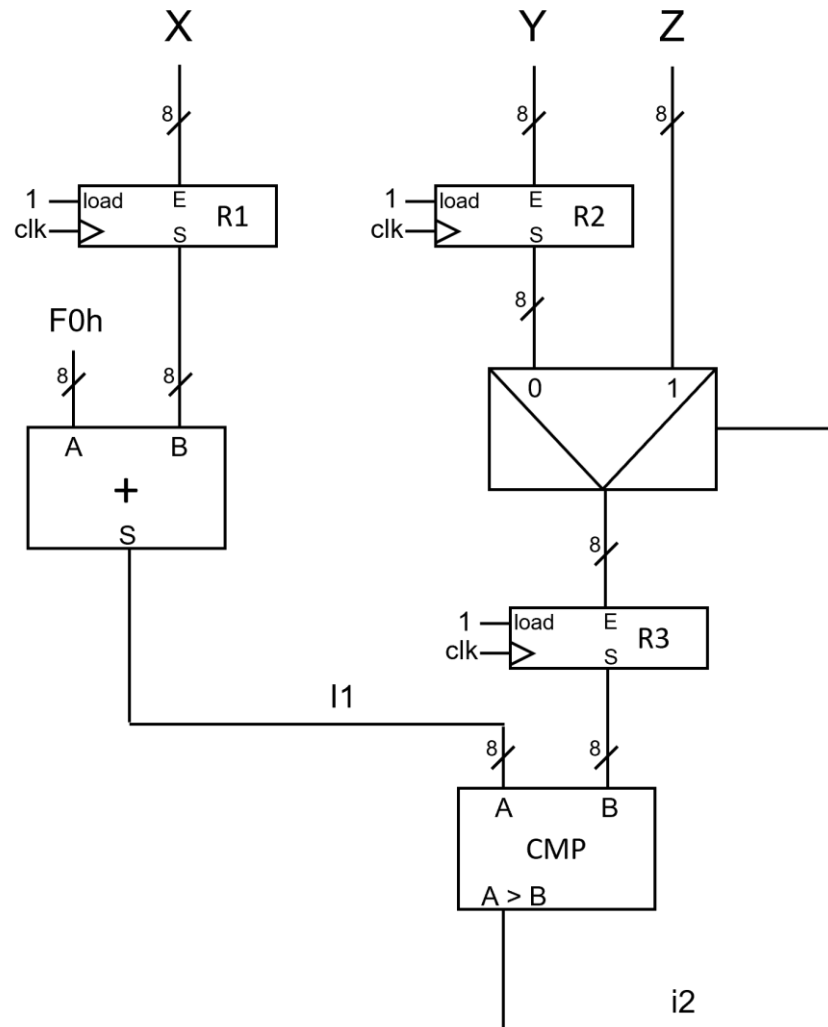
Razonando de manera análoga rellenamos todo el cronograma, que queda tal y como se muestra a continuación.



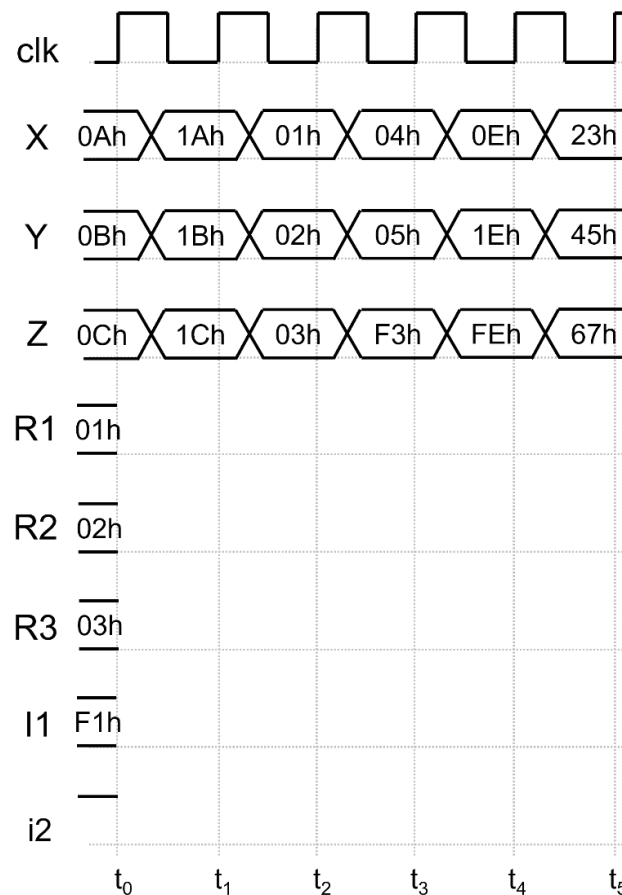


Ejercicio 4 [25%]

Dado el circuito siguiente:



Completad el cronograma siguiente, escribiendo los valores en hexadecimal (excepto la señal *i2*).



Nota: Tenéis disponible este ejercicio en VerilCHART.

Para rellenar el cronograma miramos en qué momentos se cargan los registros y con qué valores.

En cuanto a los momentos, vemos que se cargan a cada flanco de reloj, porque en la entrada *load* de los tres registros hay conectado un 1.

En cuanto a los valores que se cargan, vemos que a la entrada de R1 y de R2 llegan las señales X e Y respectivamente. Por lo tanto, podemos escribir ya de entrada todos sus valores en el cronograma (en cada flanco se escribe en ellos el valor de X o de Y en el instante justo anterior al flanco).

Lo que se cargue en R3 en cada flanco depende de cuánto valga la señal *i2*. En concreto, si $i2 = 0$ se cargará R2, y si $i2 = 1$ se cargará Z.

La señal *i2* está conectada a la salida $A > B$ de un comparador, al que llegan los valores *I1* y R3 a las entradas A y B respectivamente. Por lo tanto, $i2 = (I1 > R3)$.

Puesto que $I1 = R1 + F0h$, podemos escribir todos los valores de *I1* ya de entrada en el cronograma.

Veamos qué pasa en el instante t_0 . Justo antes de este instante, $i2 = 1$ y $Z = 0Ch$, de forma que en este flanco en R3 se escribirá 0Ch. En cuanto a R1 y R2, ya hemos escrito antes en el cronograma que en este instante se cargan respectivamente los valores 0Ah y 0Bh. Todo ello provoca que, a partir del instante t_0 , *I1* valga $F0h + 0Ah = FAh$, que es un valor mayor que 0Ch. Por lo tanto, la señal *i2* seguirá valiendo 1 como mínimo hasta el instante t_1 .



En el instante t_1 , por lo tanto, en R3 se escribirá de nuevo Z, que justo antes del flanco vale 1Ch. Y, a partir de t_1 , el valor de I1 pasará a ser $F0h + 1Ah = 0Ah$. Puesto que es un valor menor que 1Ch, entre t_1 y t_2 la señal $i2$ valdrá 0.

Razonando de manera análoga rellenamos todo el cronograma, que queda tal y como se muestra a continuación.

